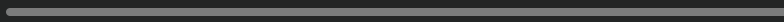




AI ENGINEERING INSIDER PODCAST
Designing a Unified Feature S...

0:00



23:33



[Listen now](#)

Designing a Unified Feature Store and Model Serving Platform

How to answer this AI system design interview question like a senior ML engineer

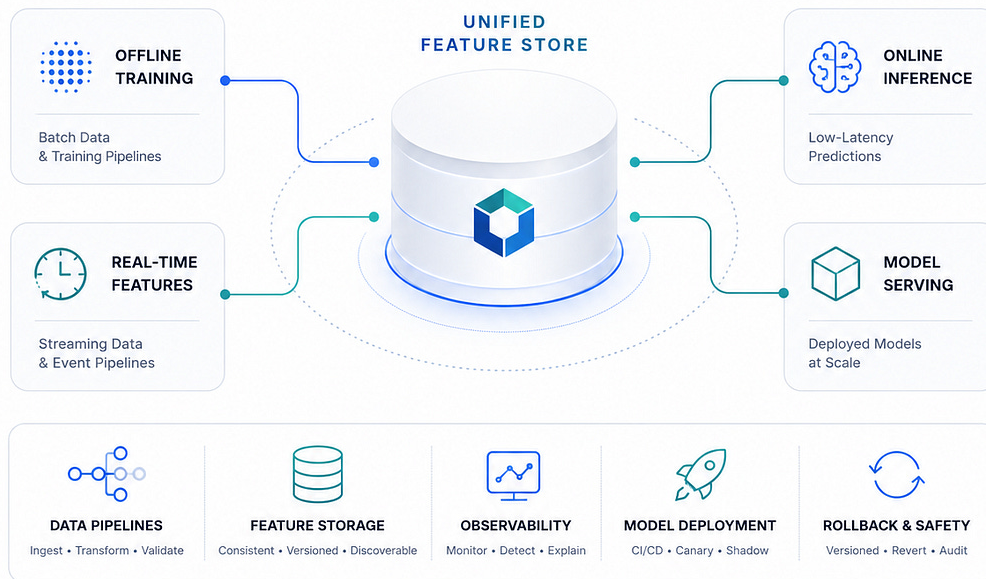


AI ENGINEERING INSIDER
JUN 15, 2026

AI SYSTEM DESIGN

Unified Feature Store

& Model Serving Platform



PODCAST

A unified feature store and model serving platform is one of the most important infrastructure components in modern machine learning systems. The core goal is simple: make features consistent, reusable, observable, and reliable across the full ML lifecycle.

In many companies, ML teams start by building features inside notebooks, batch pipelines, or application code. Over time, this creates duplicated logic, inconsistent definitions, and one of the most dangerous ML production failures: **training-serving skew**.

Training-serving skew happens when the features used during offline model training are not exactly the same as the features used during online inference. For example, a fraud model may be trained using a 30-day transaction count computed from historical batch data, but in production the online system computes that count using slightly different filtering logic. The model may perform well offline but fail silently in production.

A unified feature store solves this by allowing teams to define features once, then reuse them consistently for offline training and online inference.

1. Problem Statement

Design a multi-tenant ML platform that provides:

- ↳ A centralized feature store
- ↳ Offline feature generation for training
- ↳ Online low-latency feature serving
- ↳ Point-in-time correct training datasets
- ↳ Model deployment and serving
- ↳ Canary rollout, rollback, and observability
- ↳ Governance, access control, and feature reuse

The platform should support multiple ML use cases such as fraud detection, recommendation systems, dynamic pricing, search ranking, churn prediction, and personalization.

The most important design principle is:

Features should be defined once and served consistently everywhere.

2. Functional Requirements

The platform must support the following capabilities:

↳ **Feature definition**

ML engineers should define features using reusable declarations, such as entity keys, source tables, transformation logic, freshness requirements, and ownership metadata.

↳ **Offline feature store**

The system should store historical feature values for model training, backtesting, and experimentation.

↳ **Online feature store**

The system should serve fresh feature values at low latency for real-time inference.

↳ **Point-in-time correctness**

Training data must only use feature values that were available at the prediction timestamp. This prevents future data leakage.

↳ **Model serving**

The platform should support real-time inference, batch inference, model versioning, canary rollouts, A/B tests, and instant rollback.

↳ **Monitoring and observability**

Teams need visibility into feature freshness, null rates, drift, model latency, error rate, prediction distribution, and business metrics.

3. Non-Functional Requirements

A strong interview answer should always cover system constraints.

↳ **Low latency**

Online feature reads should typically complete in single-digit to low double-digit milliseconds, depending on the use case.

↳ **High availability**

The online serving path must remain available even if offline pipelines fail.

↳ **Scalability**

The system should support thousands of features, many teams, high QPS inference traffic, and large historical datasets.

↳ **Consistency**

Feature definitions must be versioned and shared across offline and online systems.

↳ **Security and governance**

The platform must enforce tenant isolation, access control, PII handling, lineage tracking, and audit logs.

4. High-Level Architecture

A production-grade system can be divided into five major layers:

A. Data Sources

Raw data comes from multiple systems:

- ↳ Transaction logs
- ↳ User activity events
- ↳ Application databases
- ↳ Data warehouse tables
- ↳ Streaming platforms

- ↳ Third-party signals
- ↳ Real-time application events

Examples include Kafka, Pub/Sub, Snowflake, BigQuery, PostgreSQL, S3, and data lake tables.

B. Feature Engineering Layer

This layer transforms raw data into reusable ML features.

There are two main paths:

- ↳ **Batch feature pipelines**

Used for historical aggregations, daily user statistics, long-window features, and training datasets.

- ↳ **Streaming feature pipelines**

Used for real-time signals such as current session behavior, recent transaction velocity, or latest user interactions.

Feature transformations should be defined declaratively when possible. For example:

- ↳ Entity: user_id
- ↳ Feature: user_30d_purchase_count
- ↳ Source: transactions table
- ↳ Logic: count purchases in previous 30 days
- ↳ Freshness SLA: updated hourly
- ↳ Owner: personalization ML team

The key idea is that the same feature definition should drive both offline materialization and online serving.

C. Offline Feature Store

The offline store keeps historical feature values for training and experimentation.

It is optimized for:

- ↳ Large-scale scans
- ↳ Historical joins
- ↳ Backfills
- ↳ Model training datasets
- ↳ Experiment reproducibility

Common storage choices include data warehouses, lakehouses, or object storage formats such as Parquet, Iceberg, Delta Lake, or Hudi.

The most critical requirement here is **point-in-time correctness**.

For each training example, the system must join only the feature values that existed before the prediction timestamp.

Example:

If we train a fraud model on a transaction from March 10 at 2:00 PM, we cannot use a feature value updated on March 11. That would leak future information into training.

D. Online Feature Store

The online store serves the latest feature values during real-time inference.

It is optimized for:

- ↳ Low-latency key-value lookups
- ↳ High QPS
- ↳ Fresh feature reads
- ↳ High availability

Typical storage systems include Redis, DynamoDB, Cassandra, Bigtable, or RocksDB-based serving layers.

For example, when a user opens an app, the recommendation model may request:

- ↳ User embedding
- ↳ Recent click count
- ↳ Session intent features
- ↳ Device type
- ↳ Location bucket
- ↳ Candidate item features

These features must be fetched quickly before the model can generate predictions.

5. Model Serving Layer

The model serving layer exposes trained models to production applications.

It should support:

- ↳ Online inference APIs
- ↳ Batch prediction jobs
- ↳ Model registry integration
- ↳ Versioned deployments
- ↳ Canary rollout
- ↳ Shadow testing

- ↳ A/B testing
- ↳ Rollback
- ↳ Autoscaling

A typical serving flow looks like this:

1. Application sends prediction request
2. Model server extracts entity IDs
3. Feature service retrieves online features
4. Model server applies preprocessing
5. Model generates prediction
6. Decision service applies business rules
7. Logs are written for monitoring and future training

For high-scale systems, feature retrieval and model inference should be optimized together. Feature latency often dominates model serving latency.

6. Eliminating Training-Serving Skew

This is the heart of the design.

The platform should prevent skew through:

- ↳ Shared feature definitions
- ↳ Versioned transformation logic
- ↳ Offline-online materialization from the same source
- ↳ Feature validation tests
- ↳ Schema checks
- ↳ Consistent preprocessing libraries
- ↳ Training dataset reproducibility
- ↳ Online/offline feature parity checks

A strong answer should emphasize that training-serving skew is not just a data problem. It is a platform, governance, and operational problem.

7. Multi-Tenancy and Governance

In large companies, many teams use the same ML platform. The system must support multi-tenancy.

Important capabilities include:

- ↳ Namespace separation by team or project
- ↳ Feature ownership metadata
- ↳ Access control for sensitive features
- ↳ PII masking and encryption
- ↳ Feature lineage
- ↳ Audit logs
- ↳ Cost attribution
- ↳ Deprecation workflows

For example, a payments team should not freely access private healthcare-related features unless authorized.

Feature reuse is valuable, but uncontrolled reuse creates privacy and compliance risks.

8. Monitoring and Observability

The platform should monitor both features and models.

Feature Monitoring

Track:

- ↳ Freshness
- ↳ Null rate
- ↳ Distribution drift
- ↳ Cardinality changes
- ↳ Schema mismatch
- ↳ Online/offline skew
- ↳ Pipeline failures

Model Monitoring

Track:

- ↳ Prediction latency
- ↳ Error rate
- ↳ Model output distribution
- ↳ Data drift
- ↳ Concept drift
- ↳ Calibration
- ↳ Business KPIs
- ↳ Canary performance

For example, if a recommendation model suddenly receives many null user embedding values, the model may still return predictions, but quality will degrade silently. Observability helps detect this early.

9. Deployment Strategy

A production ML platform should avoid risky full deployments.

Recommended deployment strategies:

- ↳ **Shadow deployment**

Run the new model in parallel without affecting users.

↳ **Canary rollout**

Send a small percentage of traffic to the new model.

↳ **A/B testing**

Compare model versions using controlled experiments.

↳ **Instant rollback**

Quickly revert to a previous stable model if latency, errors, or business metrics degrade.

The model registry should store metadata such as:

↳ Model version

↳ Training dataset

↳ Feature set version

↳ Evaluation metrics

↳ Approval status

↳ Deployment history

This makes deployments traceable and auditable.

10. Interview Trade-Offs

A senior-level answer should discuss trade-offs.

↳ **Freshness vs latency**

Real-time features are fresher but more expensive and complex.

↳ **Consistency vs availability**

Online serving systems may prioritize availability over perfect consistency.

↳ **Centralization vs flexibility**

A centralized platform improves reuse but may slow down teams if

governance is too rigid.

↳ **Batch vs streaming**

Batch pipelines are simpler and cheaper, while streaming pipelines support real-time decisions.

↳ **Feature reuse vs privacy**

Reusable features improve velocity but require strong access control.

Final Answer Summary

A unified feature store and model serving platform improves ML engineering velocity by making features reusable, consistent, observable, and production-ready. The feature store eliminates training-serving skew by defining features once and serving them through both offline and online paths. The offline store enables point-in-time correct training datasets, while the online store supports low-latency inference. The model serving layer provides versioned deployments, canary rollouts, A/B testing, rollback, and monitoring.

In an interview, the strongest answer should emphasize that this is not just a storage system. It is a full ML platform that connects data engineering, model training, real-time serving, governance, and observability into one reliable production workflow.

The core design principle is simple:

Define features once, serve them consistently, monitor them continuously, and deploy models safely.